

Practical 1.1: Setting Up the Local Development Environment

Install Docker, Git, VS Code, Python 3 and verify all tools

Day 1 · 3 hours · Resilience Academy SDI Training

Reference: crd.resilienceacademy.ac.tz | Data: Tanga & Mwanza, Tanzania

Learning Objectives:

- Install Docker, Docker Compose, Git, VS Code, and Python 3 on Ubuntu/WSL2
- Verify all tools are working correctly
- Run your first Docker container

Prerequisites

Before starting, ensure you have:

- Ubuntu 22.04 LTS (or WSL2 on Windows with Ubuntu)
- At least 8GB RAM and 20GB free disk space
- Internet connection
- Terminal/Bash access

Step 0: Install WSL2 (Windows Users Only)

Skip this step if you are already on Ubuntu or macOS. This is only for Windows users.

WSL2 (Windows Subsystem for Linux 2) provides a full Linux kernel on Windows. Docker Desktop requires it.

```
# Run in PowerShell as Administrator:

# Install WSL2 with Ubuntu (one command does everything)
wsl --install

# After restart, verify WSL2 is active:
wsl --list --verbose

# Should show: Ubuntu Running 2

# If VERSION shows 1, upgrade:
wsl --set-version Ubuntu 2
```

After installation, Windows will prompt you to create a Linux username and password. Then open the Ubuntu terminal and update:

```
sudo apt update && sudo apt upgrade -y
```

Enable virtualisation (if wsl –install fails)

```
# Run in PowerShell as Administrator:
dism.exe /online /enable-feature \
  /featurename:VirtualMachinePlatform /all /norestart
dism.exe /online /enable-feature \
  /featurename:Microsoft-Windows-Subsystem-Linux \
  /all /norestart
# Restart PC, then run: wsl --install
```

If BIOS virtualisation is disabled, enable **Intel VT-x** or **AMD-V** in your BIOS/UEFI settings.

Configure Docker Desktop for WSL2

After installing Docker Desktop (Step 2), configure WSL integration:

1. Open Docker Desktop → **Settings** → **General**
2. Check “**Use the WSL 2 based engine**”
3. Go to **Settings** → **Resources** → **WSL Integration**
4. Enable integration for your **Ubuntu** distro
5. Click **Apply & Restart**

Verify Docker works from inside WSL2 Ubuntu terminal:

```
# Open Ubuntu terminal, then:
docker --version
docker compose version
docker run hello-world
```

Step 1: Update System

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget gnupg ca-certificates lsb-release apt-transport-https
```

Step 2: Install Docker

```
# Add Docker official GPG key and repository
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine and Compose plugin
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# Allow current user to run Docker without sudo
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version
docker compose version
```

Note: After `usermod`, close and reopen your terminal for group changes to take effect.

Step 3: Install Git

```
sudo apt install -y git
git --version # should show 2.x or later

# Configure your identity
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global core.editor nano
```

Step 4: Install VS Code

```
# Download and install VS Code
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | \
  gpg --dearmor > packages.microsoft.gpg
sudo install -D -o root -g root -m 644 packages.microsoft.gpg \
  /etc/apt/keyrings/packages.microsoft.gpg

echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/packages.microsoft.gpg] \
  https://packages.microsoft.com/repos/code stable main" | \
  sudo tee /etc/apt/sources.list.d/vscode.list > /dev/null

sudo apt update && sudo apt install -y code
code --version
```

Step 5: Install Python 3 and GeoPandas

```
sudo apt install -y python3 python3-pip python3-venv

# Install essential data science packages
pip3 install geopandas pandas matplotlib requests

# Verify
python3 --version
python3 -c "import geopandas; print('GeoPandas OK:', geopandas.__version__)"
```

Step 6: Run Your First Container

```
# Test Docker is working
docker run hello-world

# Run an nginx web server on port 8080
docker run -d --name test-nginx -p 8080:80 nginx
curl http://localhost:8080
# You should see the nginx welcome page HTML

# Clean up
docker stop test-nginx
docker rm test-nginx
```

Exercise: Test the Live CRD API

```
# Query the Resilience Academy CRD API
curl "https://crd.resilienceacademy.ac.tz/api/v2/datasets/?search=mwanza" | \
  python3 -m json.tool | head -40
# How many Mwanza datasets are listed in "total"?
```

Checkpoint — Before moving on, verify:

- ✓ `docker -version` shows 24.x or later
- ✓ `docker compose version` shows 2.x
- ✓ `git -version` shows 2.x
- ✓ `python3 -version` shows 3.10+
- ✓ `docker run hello-world` prints "Hello from Docker!"
- ✓ CRD API curl returns JSON with Mwanza datasets

Take a screenshot showing all tools working. Share with the instructor.